RAFFLES INSTITUTION
H2 Computing (9569)

2026 Year 6

Chapter S4A: Web App HTTP

CONTENT

1. Web Technologies & Architectures

The World Wide Web, proposed in 1990 by Tim Berners-Lee at CERN, is built upon a set of layered technologies that work together to deliver web applications.
These layers can be understood in terms of:

- Architecture (how systems are organised)
- Content Structure (how information is represented)
- Communication Logic (how systems exchange data)
- Security & Integrity (how data is protected)

1.1 Client-Server Architecture

The Web operates on a **Client-Server Architecture**. This is a distributed model in which responsibilities are divided between client and server.

- **Client:** Typically a web browser (e.g., Chrome, Edge). It provides the user interface, initiates requests for resources, and interprets and displays responses.
- **Server:** A computer hosting web resources. It listens to incoming requests, processes requests, and sends back responses (web pages, images, data) to the client.

Communication is always initiated by the client. Server does not send data unless requested.

1.2 Content Structure: HTML

Web documents are written in **Hypertext Markup Language (HTML).** HTML uses tags to define structure (headings, paragraphs, images, forms) and provides semantic meaning to content. The "hypertext" element of HTML allows documents to link to one another using a Uniform Resource Locator (URL).

This provides non-linear navigation, linking across different servers and creating an interconnected global information system. This interlinking structure forms the "web" in World Wide Web.

1.3 Communication Logic: HTTP

The **Hypertext Transfer Protocol (HTTP)** defines the rules governing communication between client and server.

- **Request-Response Cycle:** Communication is always initiated by the client. A client sends a **Request Header** (containing methods like GET or POST), and the server returns a **Response** (containing a status code and the resource).
- **Statelessness:** HTTP is a stateless protocol. The server does not remember previous requests. Each request is processed independently. Any required state information must be included in each request. Additional mechanisms (e.g. cookies or tokens) are needed to manage user sessions.
- **Default Port:** Standard HTTP traffic is transmitted over **Port 80**.

1.4 Security & Integrity: HTTPS

As standard HTTP transmits data in **plaintext**, it can be intercepted and modified during transmission. To address this, **Hypertext Transfer Protocol Secure (HTTPS)** is used.

- **Encryption: HTTPS** uses encryption protocols (TLS/SSL) to protect confidentiality, ensure data integrity and prevent unauthorized interception.
- **Authentication:** Beyond encryption, HTTPS uses digital certificates to verify the identity of the server and prevent impersonation attacks.
- **Default Port:** Secure traffic is routed through **Port 443**.

Although modern web applications default to HTTPS, understanding **HTTP** is essential for learning the fundamental mechanics of the request-response model.

2. Uniform Resource Locator (URL)

2.1 Purpose of a URL

To request for a web document, you need to supply three pieces of information:

1. The host (IP address or domain name)
2. The port number
3. The path of the resource

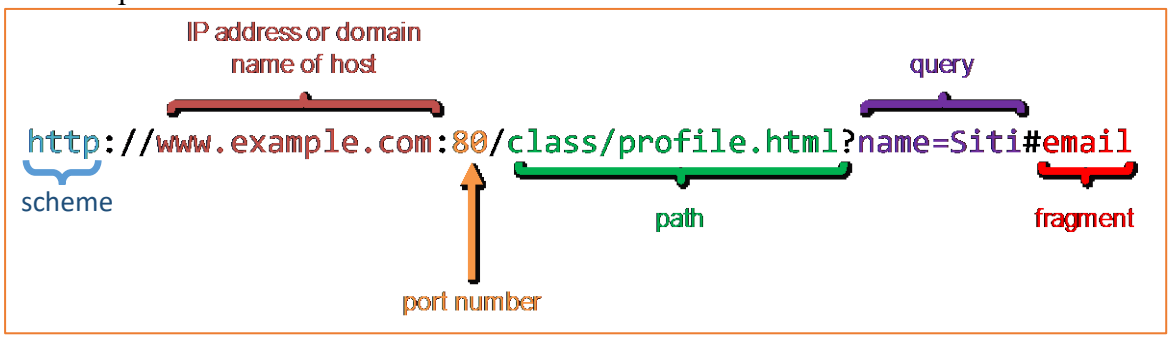
All these information are supplied to the web browser using a single string called a **Universal Resource Locator (URL)**. A URL uniquely identifies the location of resource and the protocol used to access it.

2.2 General Structure of a URL

A typical web URL has the following format:

```
scheme://host:port/path?query#fragment
```

An example of a URL:



Format: `scheme://host:port/path?query#fragment`

Component	Description
Scheme	Specifies the protocol used. Common schemes include http and https.
Host	Domain name (e.g. example.com) or IP address (e.g. 127.0.0.1)
Port (optional)	Specifies the port number on which the server is listening. Default ports for http is 80 and https is 443. If the port is not specified, the default port is used.
Path	Specifies the location of the resource on the server.
Query (optional)	Begins with ? Used to send additional data to the server. e.g. `/search?keyword=computing`
Fragment (optional)	Used to refer to a specific section within a document. e.g. `/page.html#section3` Not sent to the server. Processed only by the browser. Does not change the server's response.

2.3 Example Breakdown

Examples of URLs:

URL	Scheme	Host	Port No.	Path
`http://www.example.com`	`http`	`www.example.com`	`80`	`/`
`http://www.example.com:8080`	`http`	`www.example.com`	`8080`	`/`
`https://secure.example.com:4430/`	`https`	`secure.example.com`	`4430`	`/`
`https://example.com:80/example.com`	`https`	`example.com`	`80`	`/example.com`
`https://example.org/example.com/example.html`	`https`	`example.org`	`443`	`/example.com/example.html`
`http://complex.example.org/complex.example.com`	`http`	`complex.example.org`	`80`	`/complex.example.com`
`https://www.moe.gov.sg/news/press-releases`	`https`	`www.moe.gov.sg`	`443`	`/news/press-releases`

Examples of URLs with query and fragment components:

URL	Query	Fragment
`http://example.net:5000/com?hello=world`	`Hello=world`	`None / Omitted`
`https://sub.domain.example.org/9999/#hello-world`	`None / Omitted`	`Hello-world`
`https://example.org/res/search?keyword=computing#page-1`	`keyword=computing`	`page-1`

2.4 Percent Encoding

URLs allow only certain **unreserved characters**:

- Letters (A–Z, a–z)
- Digits (0–9)
- `-` `_` `.` `~`

All other characters must be **percent-encoded** using the format:

% + ASCII value in hexadecimal

Some percent-encodings of common characters are:

Character	*(space)*	#	%	/	:	?
ASCII Code	32	35	37	47	58	63
Percent-Encoding	%20	%23	%25	%2F	%3A	%3F

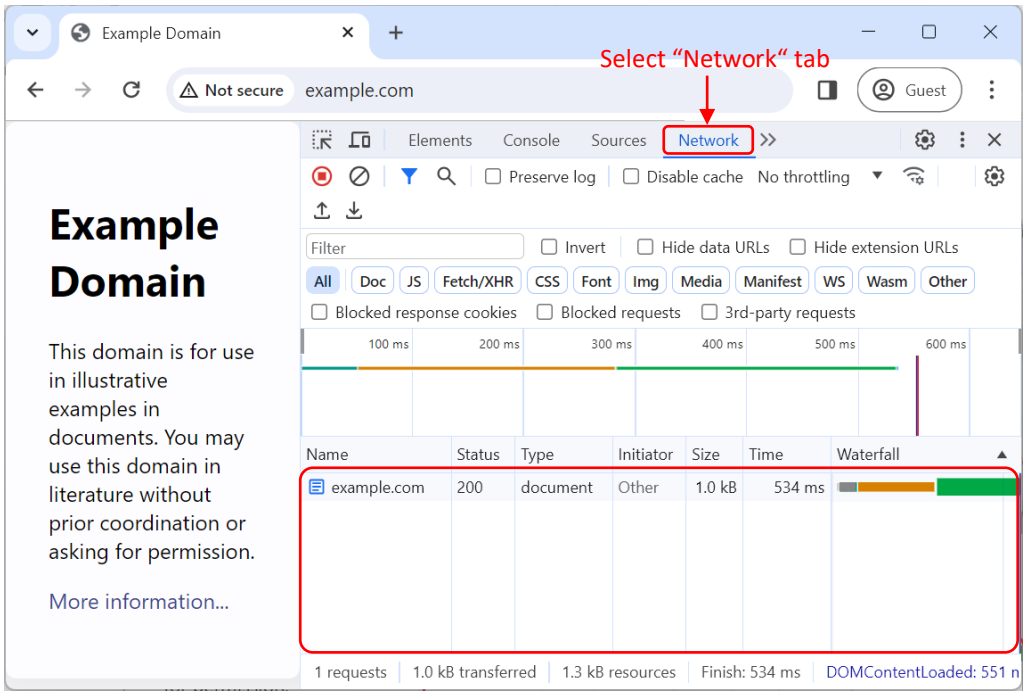
2.5 The URL Execution Process

When a URL is entered, the browser parses the **host** and **port**. For convenience, browsers allow users to omit the **protocol prefix**, though modern versions now prioritise **HTTPS** over the traditional **HTTP** default.

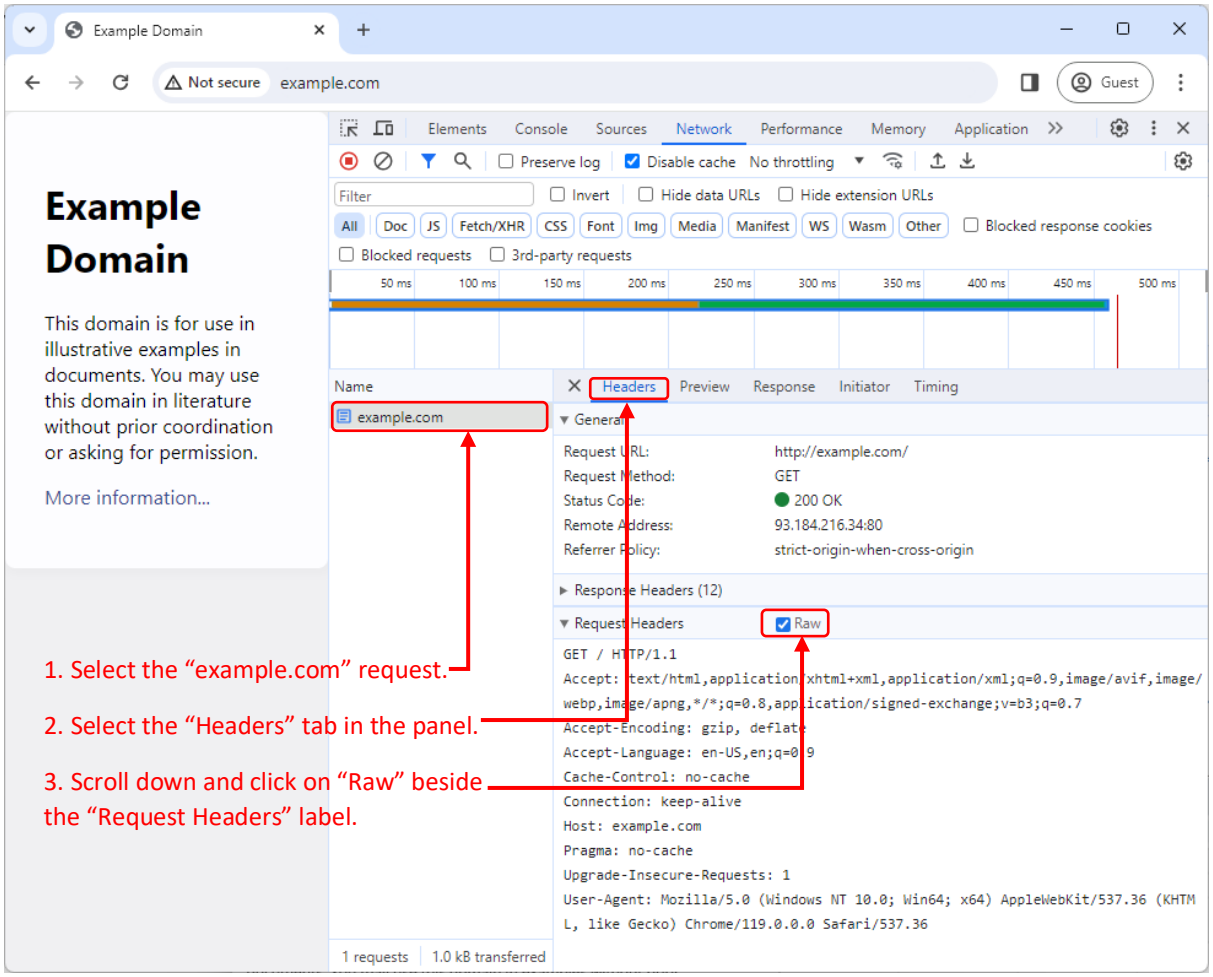
The browser then creates a **socket**, which is essentially a virtual connection point using the server's IP and port, to establish a link. Once this connection is open, the browser sends an **HTTP request** for the document at the specified **path**. The server then returns an **HTTP response** with the data. This entire exchange follows the HTTP protocol, which encodes the information as **bytes** for transmission.

3. HTTP Requests

Launch **Google Chrome** and open **Developer Tools** from the ellipsis menu under the "More tools" item. With the **Developer Tools** open, select the **Network** tab. Next, enter *http://example.com* into the address bar and press enter. After the page has loaded, the **Network** tab should list out all the HTTP requests made by the web browser. Select a request and you should be able to read more details about that request in a separate pane.



Scroll down the details pane and you should see a section labelled "**Request Headers**" with an option labelled "**Raw**" next to it. (If the "**Raw**" option is missing, make sure you are using HTTP and not HTTPS. You may also need to refresh the page.) Click on the option and you will see a portion of the bytes that were sent by your web browser to the web server using a socket.



Unlike more complex protocols, HTTP 1.0 and 1.1 are text-based, so HTTP requests and responses are still readable even when they are encoded as bytes. Each request starts with a **request line**, followed by **header fields**, followed by an empty line and an optional **message body**.

request line
```
GET / HTTP/1.1
```

header fields
```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Cache-Control: no-cache
Connection: keep-alive
Host: example.com
Pragma: no-cache
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36
```

empty line *(invisible)*

message body *(optional)*
missing as this example has no message body

3.1 Request Line

The request line contains **three important pieces of information** separated by spaces: the HTTP command (also called **method** or **verb**) that the client wants to perform, the path of the relevant document (together with the query portion of the URL, if any) and the version of HTTP used by the client.

The two most common HTTP methods are **GET** and **POST**. **GET** is used whenever we simply want to get or request for a web document without the intention to make any changes. For instance, when we enter *http://example.com/test.html* into the address bar of the web browser, the web browser actually creates a socket to the *example.com* server and sends a request with the following request line:

```
GET /test.html HTTP/1.1
```

Notice that the **path** that is used in the request line comes directly from the URL that was entered in the address bar. An exception is if the URL's path is empty, such as for the URL *http://example.com* (without a slash). In this case, a path of / is used instead so *http://example.com* (without a slash) and *http://example.com/* (with a slash) result in the same HTTP request.

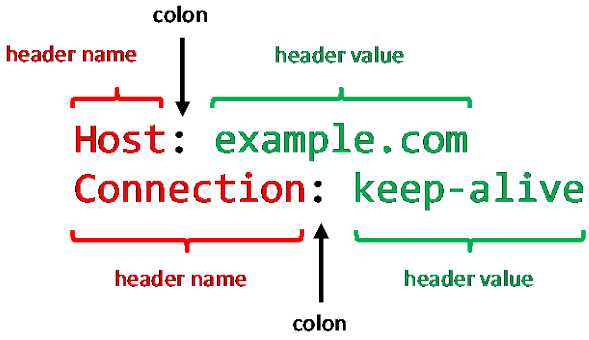
If the URL has a **query component**, it is included in the request line together with the question mark separator. For instance, when *http://example.com/hello?q=world* is entered into a web browser's address bar, the following request line is used:

```
GET /hello?q=world HTTP/1.1
```

On the other hand, **POST** is used when we intend to **post or submit data and make changes** to data on the web server. In this case, the request will usually include a message body containing the data that is being submitted. We will see examples of what a POST request looks like later in a practical task.

3.2 Header Fields

HTTP header fields provide additional information to the web server so it can customise its response accordingly. Each header field has its own line that consists of a name, followed by a colon, followed by a space and then the header field's value.



The only header field that is required by HTTP 1.1 is named **Host** and is copied directly from the host portion of the URL. This allows the same web server to provide different responses depending on which domain name is used to reach the server. For example, if *http://example.com/test.html* is entered into a web browser's address bar, the HTTP request will use the following header field:

```
Host: example.com
```

Some other important HTTP header fields are Content-Type (to specify how the message body should be interpreted) and Content-Length (to specify the message body's size in bytes). For a list of possible header fields and what they are used for, you may refer to other websites such as *https://en.wikipedia.org/wiki/List_of_HTTP_header_fields*.

3.3 Message Body

After the header fields, a HTTP request is required to have an empty line followed by an optional message body.

For **GET** requests, the message body is ignored so this portion of the request is usually omitted. However, for **POST** requests, the message body typically contains the data that is being submitted. For instance, the following is a simplified **POST** request that a browser may make after a simple web form is submitted by the user:


```
POST /send HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30

user=tim&msg=Hello%2C+World%21
```

You will learn what the above message body means and how it is related to the form shown in the next quiz.

Quiz 1

Enter the following Python program:

```
import socket

my_socket = socket.socket()
my_socket.bind(('127.0.0.1', 8000))
my_socket.listen()
new_socket, address = my_socket.accept()
received_data = new_socket.recv(1024)
print(received_data.decode())
new_socket.close()
my_socket.close()
```

This server program listens for a connection request on port number 8000, creates a connection to the client when a request is detected, and then prints out up to 1024 bytes of the request received from the client on the Python shell window. (Note that this server does NOT send any bytes back to the client – the browser.)

a) Restart the server and visit *`http://127.0.0.1:8000/`* using a browser. What is the first line of what the browser sends to the server after connecting?

---

b) Restart the server and visit *`http://127.0.0.1:8000/example`* using a browser. What is the first line of what the browser sends to the server after connecting?

---

c) Restart the server and visit *`http://127.0.0.1:8000/example?q=query`* using a browser. What is the first line of what the browser sends to the server after connecting?

---

d) Restart the server and visit *`http://127.0.0.1:8000/example#fragment`* using a browser. What is the first line of what the browser sends to the server after connecting?

---

e) The previous server program does not send any bytes back to the client before closing the socket. What do you see displayed on the browser?

---

f) What three pieces of information must the first line of a HTTP request contain?

Quiz 2

When a URL is entered into the address bar of a web browser, a HTTP request is constructed and sent to the host that is specified in the URL. Predict the request line and host header field portions of the HTTP request for each of the following URLs.

Example: `http://www.example.com/home?hello#world`

```
GET /home?hello HTTP/1.1
Host: www.example.com
```

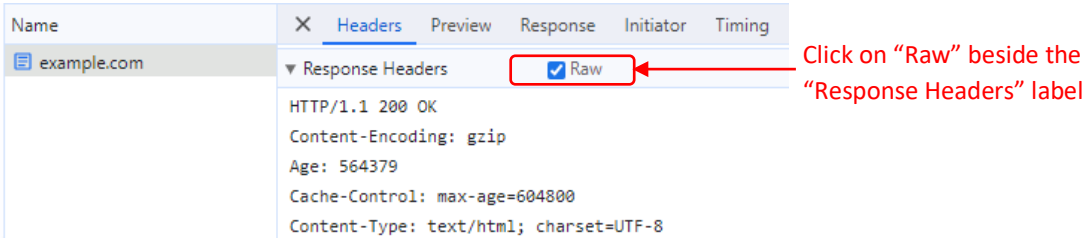
a) `http://example.net:5000/com?hello=world`

b) `http://sub.example.org/9999?keyword=computing#page-1`

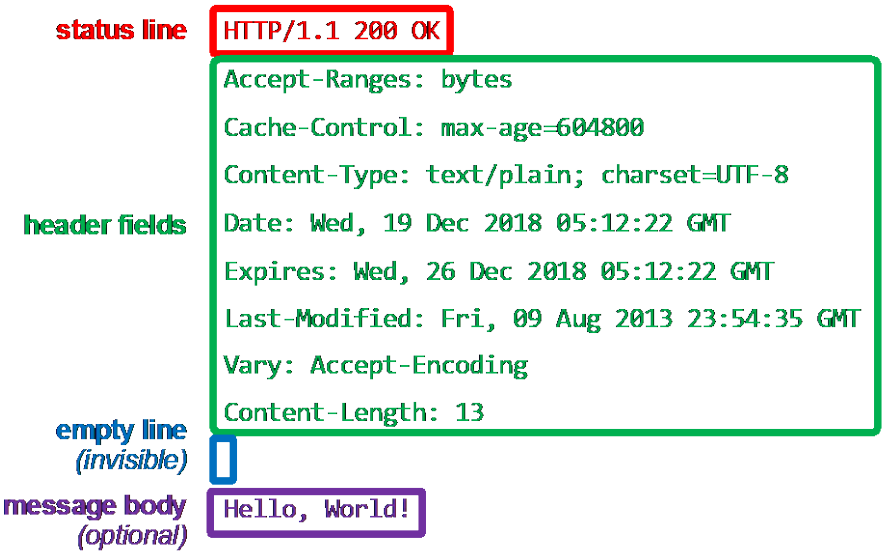
c) `http://www.moe.gov.sg#footer`

4. HTTP Responses

After a **web server** receives a HTTP request, it **sends back a HTTP response** using the same socket. Open Google Chrome's Developer Tools, select the "Network" tab and visit *http://example.com* again. Above the section labelled "Request Headers" should be another section labelled "Response Headers". Look for a "Raw" option next to this label and click on it. (Once again, if the "Raw" option is missing, make sure you are using HTTP and not HTTPS.)



HTTP responses have a similar format to HTTP requests. Each response starts with a **status line**, followed by **header fields**, followed by an empty line and an optional **message body**.



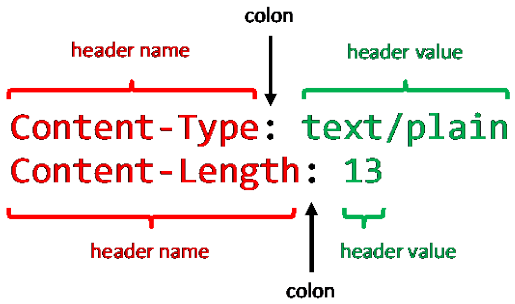
4.1 Status Line

The status line summarises if the web server was able to perform the request that was received from the client or web browser. It contains three important pieces of information separated by spaces: the version of HTTP that the server uses, a **status code** and a **reason phrase**. Each status code has a recommended corresponding reason phrase that helps to explain what the status code means. Three common status codes and their corresponding reason phrases are:

Status Code	200	404	500
Reason Phrase	OK	Not Found	Internal Server Error

You may find out all the status codes and their reason phrases on other websites such as *https://en.wikipedia.org/wiki/List_of_HTTP_status_codes*.

4.2 Header Fields



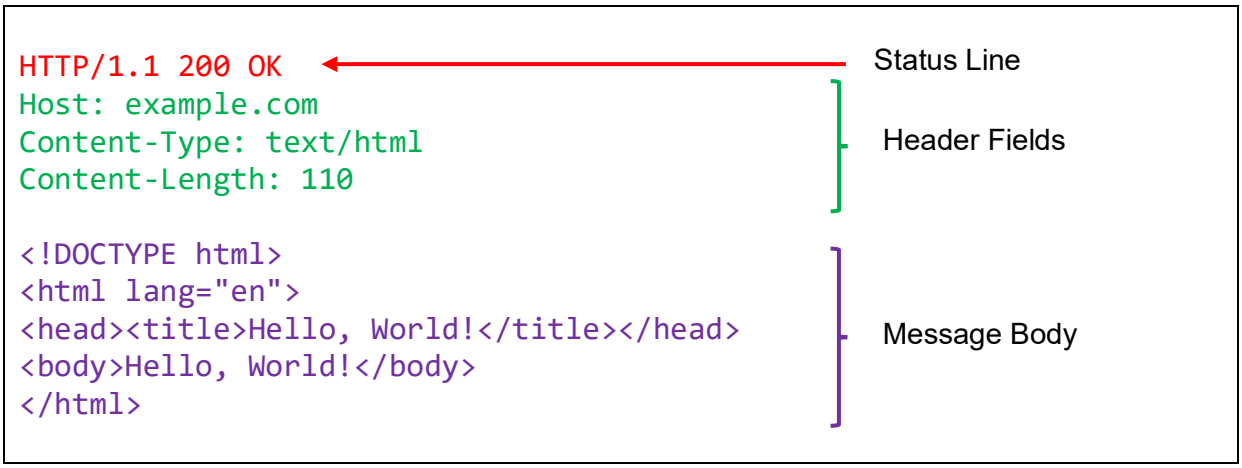
Like HTTP requests, HTTP responses also have header fields that provide additional information. Each header field has its own line that consists of a name, followed by a colon, followed by a space and then the header field's value.

A typical HTTP response would have a Content-Type value to specify how the message body should be interpreted as well as a Content-Length value to specify the message body's size in bytes. Some typical values for Content-Type value are *text/plain*, *text/html*, *text/css*, *image/png*, *image/gif* and *image/jpg*.

4.3 Message Body

Like HTTP requests, the header fields and message body for a HTTP request are separated by an empty line.

The message body will typically contain a document in the format specified by the Content-Type header field. For instance, the following is a simplified HTTP response that a browser may make after receiving a request for a HTML document:



The message body in the above response is an example of a HTML document.

Quiz 3

Enter the following program:

```
1   import socket
2
3   response = b'HTTP/1.1 200 OK\r\n'
4   response += b'Content-Type: text/plain\r\n'
5   response += b'Content-Length: 13\r\n'
6   response += b'\r\n'
7   response += b'Hello,\nWorld!'
8
9   my_socket = socket.socket()
10  my_socket.bind(('127.0.0.1', 8000))
11  my_socket.listen()
12  new_socket, address = my_socket.accept()
13  new_socket.sendall(response)
14  input('Press enter to quit: ')
15  new_socket.close()
16  my_socket.close()
```

Unlike the previous server program that did not send anything back to the browser, this server sends back a hard-coded HTTP response.

Notice from lines 3 to 7 that HTTP uses `'\r\n'` for line endings instead of `'\n'` such that every line that comes before the message body (including the empty line) ends with `'\r\n'` instead of `'\n'`. `'\r'` is called the carriage return character and `'\n'` is called the line feed character. Historically, when computers used printers for output, both characters were needed to position the print head correctly for the next line. Some systems such as HTTP still require use of these two characters to denote the end of a line.

However, note that the requirement to use `\r\n` only applies to the headers and does not affect the message body. HTTP transmits the message body without trying to interpret it, so it can use `'\n'` for line endings if desired.

a) Visit *`http://127.0.0.1:8000`* on a browser. What do you see displayed on the browser?

b) What three pieces of information must the first line of a HTTP response contain?

Quiz 4

1. What is the primary purpose of HTTP headers in an HTTP request?

---

2. Which HTTP method is typically used when a web browser wants to retrieve a document without making any changes to the server data?

---

3. What does HTTPS stand for?

---

4. Which HTTP response status code indicates that the requested resource was not found on the server?

---

5. What is the default port number for HTTP?

---

6. What does the acronym "URL" stand for?

---

7. Which component of a URL specifies the protocol being used (such as HTTP or HTTPS)?

---

8. Which HTTP method is typically used when submitting a form on a web page?

---

9. What is the purpose of percent-encoding in URLs?

---

10. What is the significance of the fragment component in a URL?

Quiz 5

Explain what the following Python program does.

```
import socket

my_socket = socket.socket()
my_socket.bind(('127.0.0.1', 8000))
my_socket.listen()
new_socket, address = my_socket.accept()
received_data = new_socket.recv(1024)
request_lines = received_data.decode().split('\r\n')
print("First line of HTTP request:", request_lines[0])
new_socket.close()
my_socket.close()
```